Advanced Terraform on AWS Lab6b Pipeline Automation

Expected Duration 2

Prerequisites 2

Part 1. Local PS Pipeline-Enforced Changes 2

- Teaching Points 2
- Phase 1. Review the Promotion Model 3
- Phase 2. Review the Local Pipeline 4
- Phase 3. Pipeline-Applied Changes 1 4
- Phase 4. Pipeline-Applied Changes 2 4
- Phase 5. Verify Environment Order Enforcement 5
- Phase 6. Promote Through the Pipeline 5

Part 2. Automated Promotion with GitActions Pipelines 6

- Phase 1. Create the env-roots GitHub Repository 6
- Phase 2. Copy DEV, TEST and PROD Environment Roots 7
- Phase 3. Configure GitHub Environments and Approvals 9
- Phase 4. Add the change-all Workflow 10
- Phase 5. Establish Trust Between GitHub Actions and AWS 12
- Phase 6. Release Network Module v1.0.2 15
- Phase 7. Promote Network Module v1.0.2 to DEV 16
- Phase 8. Attempt an Out-of-Order PROD Promotion 17
- Phase 9 – Promote Network Module v1.0.2 Through TEST and PROD 19

Part 3. Environment-Scoped Promotion Pipelines 20

- Phase 1 – Rethink Promotion Responsibility 20
- Phase 2 – Add Environment-Scoped PR Plan Pipelines 22
- Phase 3 – Remove the Centralised Deployment Workflow 27
- Phase 4 – Add Environment-Scoped Change Pipelines 28
- Phase 5 – End-to-End Environment-Scoped Promotion 32

Phase 6 – Reflection..... 34

Expected Duration

Part 1 of this lab is expected to take between 20-30 minutes
Part 2 of this lab is expected to take 40-50 minutes
Part 3 of this lab is expected to take 40-50 minutes
Part 4 of this lab is expected to take 3-5 minutes

Total expected lab time is therefore between 103 - 135 minutes

Take breaks between parts as needed.

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Prerequisites

This lab requires that Lab6a has been completed successfully.

Part 1. Local PS Pipeline-Enforced Changes

Teaching Points

In Lab6a you established a promotion model based on version-pinned modules.

Promotion was represented by changing the version of a module consumed by an environment root. You deliberately controlled when environments adopted new module versions and ensured that DEV, TEST and PROD remained independent deployment targets.

Until now, all Terraform execution has been performed manually. When a version change was made, you manually executed terraform init, terraform plan and terraform apply for the target environment.

In this part of the lab, we introduce the concept of a pipeline.

At this stage, a pipeline does not mean GitHub Actions, Jenkins, Azure DevOps, GitLab CI/CD or any other automation platform.

A pipeline simply means: A repeatable sequence of commands executed by a machine instead of a human.

The objective of pipelining is not to change how promotion decisions are made. The objective is to ensure that once a decision has been made, execution occurs consistently, predictably and according to agreed rules.

PowerShell scripts have been supplied that act as a simple local pipeline. They:

- Execute Terraform in a fixed sequence
- Enforce environment promotion rules
- Prevent out-of-order promotions
- Remove the need for manual Terraform execution
- Produce consistent execution behaviour

They do not:

- Decide when promotion should occur
- Modify module versions
- Automatically promote environments
- Replace human approval

The distinction is important. Humans decide whether an environment should adopt a new module version. Pipelines enforce how that decision is executed.

By the end of this part of the lab, you will understand:

- The difference between promotion decisions and promotion execution
- How automation can prevent accidental environment drift
- Why promotion order matters
- How environment guardrails can be enforced automatically
- How local automation evolves naturally into CI/CD platforms such as GitHub Actions

This part of the lab is organised into the following phases:

Phase 1 – Review the Promotion Model
Phase 2 – Review the Local Pipeline
Phase 3 – Pipeline-Applied Deployment
Phase 4 – Verify Environment Order Enforcement
Phase 5 – Promote Through the Pipeline

Phase 1. Review the Promotion Model

Before introducing automation, the following rules have been established ….

- Promotion is a module version pin change
- DEV validates new module versions
- TEST cannot advance ahead of DEV
- PROD cannot advance ahead of TEST
- Promotion also includes reverting a module version pin

Phase 2. Review the Local Pipeline

In this phase, you introduce a **local pipeline script**.

This script replaces you typing Terraform commands by hand.

The pipeline will:

- Run Terraform in a fixed order
- Enforce promotion rules
- Refuse to run if rules are violated
- Apply Terraform non-interactively

The pipeline does **not**:

- Decide when to promote
- Change module versions
- Auto-promote environments
- Modify Terraform structure

You have been given a set of PowerShell scripts at **artifacts/pipelines/powershell.** These scripts replace manual terraform init / plan / apply operations.

Phase 3. Pipeline-Applied Changes 1

A decision has been made to promote network module version **v1.0.0** in all environments.

1. Update the pinned version of the **network** module to **v1.0.0** in **main.tf** of each of the environments
2. Open a new terminal session and run each of these commands in turn ...

   c:\qatfadvaws\artifacts\pipelines\powershell\change-dev.ps1
   c:\qatfadvaws\artifacts\pipelines\powershell\change-test.ps1
   c:\qatfadvaws\artifacts\pipelines\powershell\change-prod.ps1
3. Observe:

   Terraform runs without manual input
   Commands execute in a fixed order

Phase 4. Pipeline-Applied Changes 2

A decision has been made to promote **network** module version **v1.0.1** to the **DEV** environment.

4. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **dev** environments

5. Run …

```
c:\qatfadvaws\artifacts\pipelines\powershell\change-dev.ps1
```

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

deployment_summary = {
  "environment" = "DEV"
  "modules" = {
    "compute" = "1.0.0"
    "network" = "1.0.1"
  }
}
```

Phase 5. Verify Environment Order Enforcement

In this phase, you observe how the pipeline prevents unsafe promotion by trying to promote v1.0.1 of the network module to **PROD**, bypassing **TEST**.

6. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **prod** environments
7. Run …

```
c:\qatfadvaws\artifacts\pipelines\powershell\change-prod.ps1
```

8. Observe:
   - The pipeline fails
   - Terraform does not run
   - A clear error message is displayed

```
OperationStopped: (Promotion block...lign versions).:Strir
Promotion blocked: prod (1.0.1) is ahead of test (1.0.0).
```

9. This demonstrates that the pipeline enforces rules — it does not assume intent.

Phase 6. Promote Through the Pipeline

10. Promote network module v1.0.1 to TEST and then PROD using the pipeline.

Takeaways

- Pipelines enforce behaviour — they do not decide promotion
- Automation protects humans from mistakes
- Promotion remains intentional and auditable
- Terraform architecture did not change
- This model scales directly into CI/CD systems

Up to this point, the pipeline exists only on your workstation.

In the next part of the lab, you will move the same promotion logic into a shared GitHub Actions workflow, allowing promotion rules to be enforced centrally rather than relying on individual operators.

Part 2. Automated Promotion with GitActions Pipelines

Phase 1 – Create the env-roots GitHub Repository
Phase 2 – Copy DEV, TEST and PROD Environment Roots
Phase 3 – Configure GitHub Environments and Approvals
Phase 4 – Add the change-all Workflow
Phase 5 – Establish Trust Between GitHub Actions and AWS
Phase 6 – Release Network Module v1.0.2
Phase 7 – Promote Network v1.0.2 to DEV
Phase 8 – Attempt an Out-of-Order PROD Promotion
Phase 9 – Promote Network v1.0.2 Through TEST and PROD

Phase 1. Create the env-roots GitHub Repository

In this phase, you create the GitHub repository that will hold the environment roots for DEV, TEST and PROD. This repository becomes the central location from which environment configuration is managed, reviewed and promoted. It will act as the source of truth for all future environment changes.

In Lab6a, you published the network and compute modules to separate GitHub repositories. Those repositories contain reusable module code. In this phase, you create a third repository that will contain the environment root configurations for DEV, TEST and PROD.

This separation is important:

module repositories	reusable building blocks
env-roots repository	environment configuration and promotion decisions

1. In GitHub, create a new repository named **env-roots**

   The module repositories created in Lab6a contain reusable Terraform modules. The repository created in this phase contains environment configuration and promotion decisions. Keeping these concerns separate makes promotion behaviour easier to understand and automate.

2. Create the repository as an empty repository.

   Do not add:

README
.gitignore
license

3. In VS Code, open a PowerShell terminal.
4. Create a local folder for the environment root repository:

   cd c:
   mkdir pipeline-repos
   cd pipeline-repos

5. Clone your new empty repository:

   git clone https://github.com/<your-github-username>/env-roots.git

   For example:

   git clone https://github.com/testuser2697/env-roots.git

6. Change into the cloned repository:

   cd env-roots

7. Configure your local Git identity for this repository:

   git config --local user.name "<your-github-username>"
   git config --local user.email "<your-email-address>"

8. Add the repository folder to VS Code Explorer:

   File > Add Folder to Workspace > C:\pipeline-repos\env-roots

At this point you have an empty GitHub repository ready to receive the DEV, TEST and PROD environment roots.

Phase 2. Copy DEV, TEST and PROD Environment Roots

In this phase, you populate the repository with the environment roots created in previous labs. These roots continue to represent independent deployment targets. Although execution will soon be automated, the underlying Terraform architecture remains unchanged.

The objective is to place the environment configuration under source control so that future promotion decisions can be tracked, reviewed and automated.

Copy the environment roots

1. Copy the entire **lab6\guided\envs** folder into **env-roots**

   The resulting structure should resemble:



Create a .gitignore file

Terraform creates local files that should not be committed to source control.

2. In the root of the env-roots repository, create a **.gitignore** file
3. Populate it with:

   .terraform/
   *.tfstate
   .tfstate.
   crash.log

   Why are .tfvars files not ignored?

   In this course, .tfvars files remain under source control.

   The environment configuration they contain forms part of the promotion decision and should therefore be visible, reviewable and auditable.

Commit the repository contents

4. From a PowerShell terminal:

   ```
   cd c:\pipeline-repos\env-roots
   git add .
   git commit -m "Add environment roots (dev/test/prod)"
   git push -u origin main
   ```

Verify the GitHub repository

5. Refresh the GitHub repository page.
6. Confirm the following are visible:

   envs/dev
   envs/test
   envs/prod

7. Browse into each environment folder and verify the Terraform configuration has been uploaded successfully.

Checkpoint

At this point Github contains :

Network Module Repo
Compute Module Repo
Environment Roots Repo

The module repositories contain reusable Terraform modules.

The environment roots repository contains the environment-specific configuration that decides which module versions each environment consumes.

In the next phase, GitHub Actions will be given permission to interact with AWS so that Terraform execution can be automated.

Phase 3. Configure GitHub Environments and Approvals

In this phase, you create DEV, TEST and PROD environments within GitHub and introduce approval controls. These environments provide governance over promotion, ensuring that changes remain intentional and visible.

Although all environments use the same Terraform architecture, they do not share the same level of control. DEV is intended for rapid validation, while TEST and PROD introduce increasing levels of oversight.

GitHub Environments provide a mechanism for associating approvals and deployment history with a specific target environment. By the end of this phase, GitHub will be capable of enforcing approval requirements before a workflow is permitted to deploy to TEST or PROD.

Create the DEV Environment

1. Open the **env-roots** repository.
2. Navigate to: **Settings > Environments**
3. Click: **New Environment**
4. Create an environment named: **dev**
5. Click: **Configure Environment**
6. Leave all protection rules disabled.

The DEV environment is intended to provide rapid feedback and should not require approval.

Create the TEST and Prod Environments

7. Click: New Environment

8. Create an environment named: **test**
9. Open the **TEST** environment.
10. Under Deployment Protection Rules, configure: **Required Reviewers**
11. Add **yourself** as an approver.
12. Save the protection rules.
13. Repeat these steps to create environment **prod**

Why Are We Doing This?

At this point no workflows reference these environments.

However, once a workflow targets an environment, GitHub can:

- Record deployment history
- Pause for approval
- Associate deployments with environments
- Track promotion activity

This allows GitHub to become part of the promotion process rather than simply executing Terraform.

Checkpoint

At this point:

- DEV environment exists
- TEST environment exists
- PROD environment exists
- TEST requires approval
- PROD requires approval
- GitHub is ready to participate in promotion governance

In the next phase, you will create your first GitHub Actions workflow and move the promotion logic previously implemented in PowerShell into GitHub Actions.

Phase 4. Add the change-all Workflow

In Part 1, promotion rules were enforced by a local PowerShell pipeline. In this phase, you move that same logic into GitHub Actions.

A provided 'change all' workflow is intentionally centralised. It checks the pinned module versions across DEV, TEST and PROD, blocks unsafe leap-frogging, and then runs Terraform against each environment in order.

1. Review the supplied workflow file:

c:\qatfadvaws\artifacts\pipelines\gitactions\change-all.yml

2. In your **env-roots** repository, create the GitHub Actions workflow folder:

   ```
   cd c:\pipeline-repos\env-roots
   mkdir .github
   mkdir .github\workflows
   ```

3. Copy the workflow **change-all.yml** into the repository:

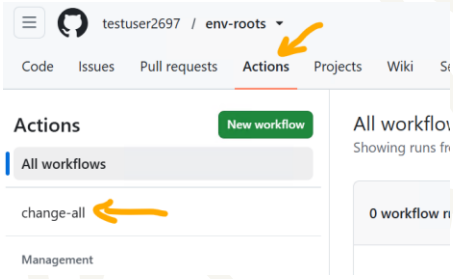
   From artifacts\pipelines\gitactions to **env-roots\.github\workflows**

4. Commit and push the workflow:

   ```
   cd c:\pipeline-repos\env-roots
   git add .github\workflows\change-all.yml
   git commit -m "Add change-all workflow"
   git push
   ```

5. Open the env-roots repository in GitHub.
6. Select **Actions**.
7. Confirm that the workflow appears:

   change-all



   Do not run the workflow manually yet.

Review the workflow behaviour

The workflow will:

Validate promotion order
↓
Apply DEV
↓
Apply TEST
↓
Apply PROD

The validation step checks that:

TEST is not ahead of DEV
PROD is not ahead of TEST

If leap-frogging is detected, the workflow fails before Terraform runs.

Checkpoint

At this point:

- change-all.yml exists in .github/workflows
- GitHub Actions can see the workflow
- The workflow uses OIDC authentication
- The workflow enforces DEV → TEST → PROD ordering

In the next phase, you will publish a new network module version for the workflow to promote.

Phase 5. Establish Trust Between GitHub Actions and AWS

Before GitHub Actions can execute Terraform, AWS must be willing to trust the identity presented by the workflow.

GitHub will request a short-lived OpenID Connect (OIDC) token each time the workflow runs. AWS examines the claims contained within that token and decides whether the workflow is permitted to assume an IAM role.

Authentication proves who GitHub claims to be. Authorisation determines whether AWS trusts that identity.

In this phase you will:

• Create an OIDC trust relationship.
• Configure GitHub Actions to request temporary AWS credentials.
• Observe an authentication failure.
• Inspect the identity claims presented by GitHub.
• Use those claims to update the IAM role trust policy.
• Successfully authenticate without storing AWS credentials.

Create the GitHub OIDC Provider

1. Navigate to IAM → Identity Providers → Add Provider.

   Provider Type: **OpenID Connect**
   Provider URL: **token.actions.githubusercontent.com**

Audience: **sts.amazonaws.com**

AWS now trusts GitHub as an identity provider, but it does not yet trust any particular GitHub repository.

Create the GitHub Actions IAM Role

2. Create a new **Web Identity** role using the GitHub OIDC provider.

   GitHub Organisation/User: **<your GitHub username>**

   Repository: **env-roots**

3. Attach the **qatip-adv_policy** managed policy supplied for this lab. (If running this lab using your own AWS account, then select/create an appropriate policy)

   Role name: **gha-terraform-env-roots-role**

   Copy the Role ARN after the role has been created as you will need it later.

Configure GitHub Repository Variables

4. Create the following repository variables:

   AWS_ROLE_ARN
   AWS_REGION
   TF_BACKEND_BUCKET

   using the values created during the previous phases.

Execute the Workflow

The supplied workflow contains an additional diagnostic step named 'Inspect OIDC Claims'.

5. Run the **change-all** workflow.

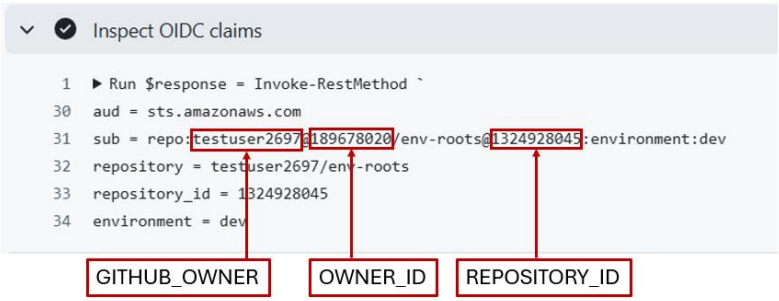
   The workflow is expected to fail during the 'Configure AWS Credentials' step with an AssumeRoleWithWebIdentity error.

   This failure is intentional and forms part of the learning exercise.

Inspect the GitHub Identity

©QA 2026

6. Open the 'Inspect OIDC Claims' step and record the following values (yours will differ):



Update the IAM Role Trust Policy

7. Open the IAM Role → Trust relationships → Edit trust policy.
8. Replace the generated trust policy with the template found at **c:\qatfadvaws\artifacts\oidc\template.json**, substituting the placeholders with your AWS account ID and values collected from the OIDC claims.

   Training use only
   *This lab deliberately uses a wildcard in the IAM role's OIDC subject condition: repo:{GITHUB_OWNER}@{OWNER_ID}/env-roots@{REPOSITORY_ID}:**
   This allows GitHub Actions jobs from different branches, pull requests and environments within the specified repository to assume the same AWS role. Each matching job receives the permissions attached to that role; a workflow labelled "plan only" for example, does not restrict those AWS permissions.
   This configuration is too broad for production. Production trust policies should restrict access to approved branches, environments and workflows, with separate least-privilege roles for planning and deployment.

Verify the Trust Relationship

9. Re-run the workflow.

   The Configure AWS Credentials step should now succeed.

   GitHub has not supplied an AWS access key or secret key. Instead:

   - GitHub requests a short-lived OIDC token.
   - AWS validates the token claims against the trust policy.
   - AWS issues temporary credentials for the workflow.

Checkpoint
By the end of this phase you should understand:

- AWS recognises GitHub's OIDC provider.

- The IAM role defines which GitHub identities are trusted.
- No AWS credentials are stored in GitHub.
- Temporary credentials are issued only after the OIDC token claims satisfy the IAM role trust policy.

Phase 6. Release Network Module v1.0.2

Before a new module version can be promoted, it must first be published. In this phase, you make a small non-breaking change to the network module, commit it, tag it, and publish it as **v1.0.2**

At this point, the new version will exist in GitHub, but no environment will consume it yet.

1. Open the network module repository in VS Code:

   C:\module-repos\terraform-aws-module-network

2. Open: **main.tf**
3. Update the module version to v1.0.2 ....

```
resource "terraform_data" "module_version" {
  input = {
    module  = "network"
    version = "1.0.2"
    change  = "network module version 1.0.2 release"
  }
}
```

   As before, this is essentially a no-op update of actual AWS resources, but it acts as a version change here. In production environments, actual resource changes would be made.

4. Tag and push the module update to Git...

   cd c:\module-repos\terraform-aws-module-network
   git status
   git add main.tf
   git commit -m "Publish network module v1.0.2"
   git tag v1.0.2
   git push origin main
   git push origin v1.0.2

5. In GitHub, open the terraform-aws-module-network repository.
6. Confirm that the v1.0.2 tag exists.

Checkpoint

At this point:

- Network module v1.0.2 has been published
- DEV, TEST and PROD still consume their currently pinned module versions
- No infrastructure has changed yet

The next phase promotes v1.0.2 to DEV.

Phase 7. Promote Network Module v1.0.2 to DEV

Promotion is performed by changing the module version pin in the environment root. In this phase, DEV is updated to consume the newly published network module version.

This change will trigger the change-all workflow. The workflow will apply DEV first, then continue through TEST and PROD. At this stage, TEST and PROD should produce no-op plans because their pinned versions have not changed.

1. Open the DEV environment root:

   c:\pipeline-repos\env-roots\envs\dev\main.tf

2. Locate the **network** module source.
3. Change the network module reference from:

   ?ref=v1.0.1 to **?ref=v1.0.2**

4. Commit and push the change:

   ```
   cd c:\pipeline-repos\env-roots
   git add .\envs\dev\main.tf
   git commit -m "Promote network module v1.0.2 to DEV"
   git push
   ```

5. Open the **env-roots** repository in GitHub.
6. Navigate to **Actions**.
7. Open the running **change-all** workflow.
8. Observe:

   Validate promotion order succeeds
   DEV applies the v1.0.2 change
   TEST waits for approval
   PROD waits for approval after TEST

© QA 2026



9. Approve TEST and PROD when prompted.
10. Verify that TEST and PROD complete without infrastructure changes.

Note: GitHub may display warnings about Node.js versions used by third-party actions. These are GitHub platform warnings and do not indicate a Terraform or AWS failure. Continue if the workflow completes successfully.

Checkpoint

At this point:

- DEV consumes network module v1.0.2
- TEST still consumes network module v1.0.1
- PROD still consumes network module v1.0.1
- change-all has demonstrated centralised execution

Phase 8. Attempt an Out-of-Order PROD Promotion

In this phase, you deliberately attempt an invalid promotion by advancing PROD ahead of TEST. The objective is not to succeed. The objective is to demonstrate that automation enforces promotion policy and prevents environments from leap-frogging one another.

The change-all workflow validates the pinned module versions before Terraform executes. If an environment attempts to move ahead of the environment before it, the workflow should fail.

Update the PROD Environment

1. Open **c:\pipeline-repos\env-roots\envs\prod\main.tf**
2. Locate the **network** module source.
3. Change the network module reference from:

   ?ref=v1.0.1 to **?ref=v1.0.2**

Commit and Push the Change

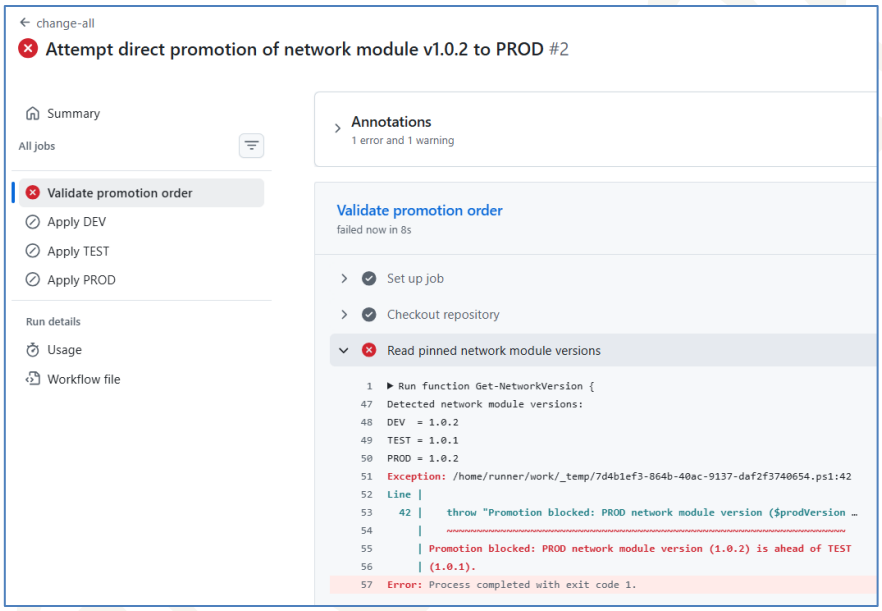
4. Commit the change:

```
cd c:\pipeline-repos\env-roots
git add .\envs\prod\main.tf
git commit -m "Attempt direct promotion of network module v1.0.2 to PROD"
git push
```

Observe the Workflow Failure

5. Open the env-roots repository in GitHub.
6. Navigate to: Actions
7. Open the running change-all workflow.
8. Observe that the workflow fails during **Validate promotion order**
9. Review the workflow output.
10. You should see a message similar to:

    Promotion blocked:
    PROD network module version (1.0.2)
    is ahead of TEST (1.0.1)



Although GitHub Actions can automate execution, it does not change the promotion rules.

Checkpoint

At this point:

- DEV consumes network module v1.0.2
- TEST consumes network module v1.0.1
- PROD consumes network module v1.0.2 (configuration only)

However:

- No PROD deployment occurred
- The workflow detected leap-frogging
- The promotion was blocked

In the next and final phase of Part 2, you will follow the correct promotion path by advancing TEST and then PROD in sequence.

Phase 9 – Promote Network Module v1.0.2 Through TEST and PROD

Having observed an invalid promotion attempt, you will now follow the correct promotion path. The failed workflow showed that PROD cannot advance ahead of TEST. To resolve this, TEST must be promoted to the same network module version before PROD can be applied successfully.

Update the TEST Environment

1. Open: **c:\pipeline-repos\env-roots\envs\test\main.tf**
2. Locate the **network** module source.
3. Change the network module reference from:

   ?ref=v1.0.1 to ?ref=v1.0.**2**

Confirm the PROD Environment

4. Open **c:\pipeline-repos\env-roots\envs\prod\main.tf**
5. Confirm that **PROD** is already pinned to: **?ref=v1.0.2**

   This change was made in the previous phase, but it was not deployed because the workflow failed before Terraform executed

Commit and Push the Correct Promotion

6. Commit the TEST update:

```
cd c:\pipeline-repos\env-roots
git add .\envs\test\main.tf
git commit -m "Promote network module v1.0.2 to DEV"
git push
```

Observe the Workflow

7. Open the **env-roots** repository in GitHub.
8. Navigate to: **Actions**
9. Open the running change-all workflow.
10. Observe...

    Validate promotion order succeeds
    DEV runs first
    TEST waits for approval

©QA 2026

PROD waits for approval after TEST

11. Approve the **TEST** deployment when prompted.
12. Approve the **PROD** deployment when prompted.
13. Confirm that the workflow completes successfully.

Verify the Final State

14. Confirm the network module versions are now:

```
DEV   network v1.0.2
TEST  network v1.0.2
PROD  network v1.0.2
```

15. Confirm the compute module remains unchanged:

```
DEV   compute v1.0.0
TEST  compute v1.0.0
PROD  compute v1.0.0
```

Takeaways

At this point:

- GitHub Actions has enforced promotion order
- DEV, TEST and PROD have been promoted in sequence
- TEST and PROD required approval
- The same promotion model used locally now runs centrally
- The compute module lifecycle remains independent of the network module

This completes the first GitHub Actions automation pattern.

Part 3. Environment-Scoped Promotion Pipelines

Phase 1 – Rethink Promotion Responsibility

In Part 2, you successfully automated Terraform execution using a single GitHub Actions workflow. That workflow enforced promotion order, respected environment approvals, and prevented unsafe changes from being deployed. It provided a simple and effective way to introduce automation into the promotion process.

The workflow operated using the following model:

Validate Promotion Order > DEV > TEST > PROD

This design successfully demonstrated:

- Promotion order enforcement
- Environment approvals
- GitHub OIDC authentication
- Automated Terraform execution
- Centralised governance

However, as environments become more independent, this approach begins to show limitations. Consider the following scenarios:

- A change is made only to the DEV environment
- A change is made only to the PROD environment
- DEV is updated daily, while PROD changes only once per month

In each case, the same workflow is still responsible for processing all environments. Although this works, it means deployment execution is being organised around the pipeline rather than around the environment being promoted.

As systems grow, environments often move at different speeds and are managed by different teams. A single deployment workflow can become increasingly difficult to maintain because it attempts to coordinate multiple independent release schedules.

A more scalable approach is to give each environment its own deployment lifecycle:

DEV > Own Workflow
TEST > Own Workflow
PROD > Own Workflow

This allows environments to advance independently while preserving the promotion model introduced in Lab 6A.

Before continuing, it is important to remember the following principle:

- Automation should not decide when promotion occurs. Automation should execute promotion once the decision has been made.
- The promotion model itself is not changing.
- Promotion is still represented by a module version pin change.
- Human operators still decide when promotion should occur.
- The only change is that deployment responsibility will move closer to the environment being promoted.

Checkpoint

Before proceeding, ensure you understand:

• Why the change-all workflow was useful

• Why centralised deployment becomes harder to manage as environments grow
• Why environments should be independently deployable
• Why promotion decisions remain a human responsibility
• Why the next phases introduce environment-scoped workflows

In the next phase, you will introduce environment-specific Terraform plan workflows that provide visibility into proposed changes before they are accepted and deployed.

Phase 2 – Add Environment-Scoped PR Plan Pipelines

Before infrastructure changes are deployed, it is often desirable to understand exactly what Terraform intends to do. This is the purpose of a plan pipeline.

A plan pipeline performs validation and planning activities but deliberately stops short of deployment. It allows reviewers to examine proposed changes while they are still under discussion and before any infrastructure is modified.

Unlike the deployment workflows introduced later in this lab, the plan pipelines do not execute terraform apply. Their purpose is to provide visibility rather than implementation.

The objective of this phase is to introduce separate pull request plan workflows for DEV, TEST and PROD. Each workflow validates only its own environment, ensuring that a change proposed for DEV does not unnecessarily trigger planning activity for TEST or PROD.

This provides a clear separation between:

Proposed Change > Terraform Plan > Review

and:

Accepted Change > Terraform Apply

By keeping planning and deployment separate, reviewers gain visibility into the impact of a change before deciding whether it should be accepted.

Create a Branch for the New Workflows

The new workflows will be added using a feature branch and merged into the repository through a pull request.

1. Open a PowerShell terminal and change into your local clone of the e**nv-roots** repository:

   cd c:\pipeline-repos\env-roots

2. Ensure your local main branch is current and you have no branch remnants:

   git checkout main
   git pull

```
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
```

3. Create a new branch:

```
git checkout -b add-pr-plan-pipelines
```

Add the PR Plan Workflows

4. Review the supplied workflow files located in:

   c:\qatfadvaws\artifacts\pipelines\gitactions

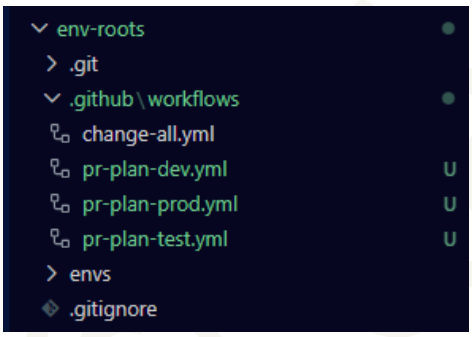
   The following files have been provided:

   pr-plan-dev.yml
   pr-plan-test.yml
   pr-plan-prod.yml

5. Copy the supplied workflow files into the existing GitHub workflows folder:

   c:\pipeline-repos\env-roots\.github\workflows

   The resulting structure should resemble:



   At this point, change-all.yml still exists. It will be retired in the next phase, after the PR plan workflows have been added and reviewed.

Commit and Push the Changes

6. Commit the workflow files:

```
git add .github\workflows\pr-plan-dev.yml
git add .github\workflows\pr-plan-test.yml
git add .github\workflows\pr-plan-prod.yml
git commit -m "Add environment PR plan workflows"
git push -u origin add-pr-plan-pipelines
```

Create a Pull Request

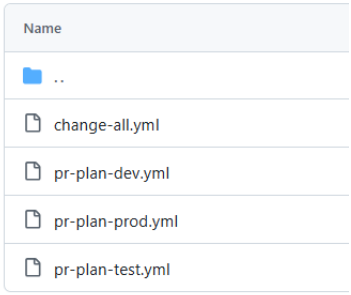
7. Open the **env-roots** repository in GitHub.

8. Create a pull request from: **add-pr-plan-pipelines** to **main**
9. Review the changes, merge the pull request and delete the branch.

Verify the Workflows

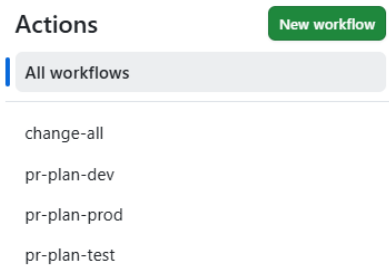
10. Refresh the repository and confirm the workflow files are visible under:

    .github/workflows



11. Navigate to: Actions
12. Confirm the following workflows are listed:

    pr-plan-dev
    pr-plan-test
    pr-plan-prod



Understanding the Workflows

- Each workflow is responsible for a single environment:

  pr-plan-dev.yml > DEV only
  pr-plan-test.yml > TEST only
  pr-plan-prod.yml > PROD only

- Each workflow performs:

  Terraform Init > Terraform Validate > Terraform Plan > STOP

- No Terraform Apply occurs.
- No infrastructure is modified.
- No state is changed.

- The workflows exist solely to provide visibility into proposed changes.

Verify PR Planning Behaviour

Before introducing environment-scoped deployment pipelines, verify that the new plan workflows execute correctly. The objective is not to propose an infrastructure change. The objective is simply to observe the planning process.

13. Create a new branch:

    ```
    cd c:\pipeline-repos\env-roots
    git checkout main
    git pull
    git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
    git checkout -b test-pr-planning
    ```

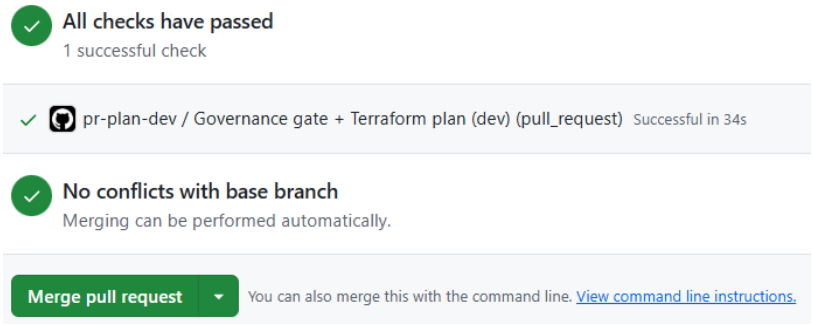
14. Open **envs/dev/main.tf**
15. Add the following comment anywhere within the file:

    ```
    # PR planning validation test
    ```

16. Commit and push the change:

    ```
    cd c:\pipeline-repos\env-roots
    git add .\envs\dev\main.tf
    git commit -m "Test DEV PR planning workflow"
    git push -u origin test-pr-planning
    ```

17. Create a Pull Request from: **test-pr-planning** to **main**
18. Open the Pull Request and observe the workflow activity.
19. Confirm **pr-plan-dev** runs automatically.
20. Confirm **pr-plan-test** does not run.
21. Confirm **pr-plan-prod** does not run.



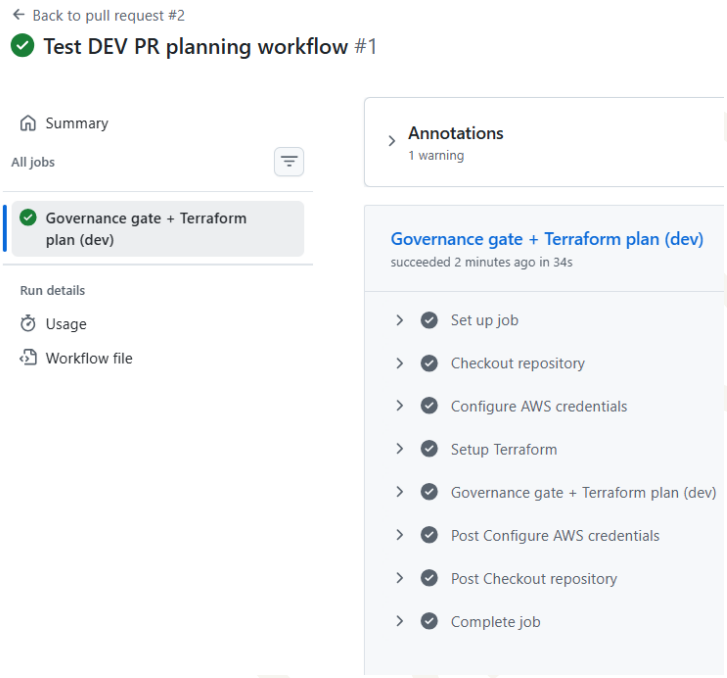
22. Review the workflow output.

You should observe:

terraform init
terraform validate
terraform plan

Because only a comment was added, Terraform should report:

No changes.
Infrastructure is up-to-date.



23. Because only a comment was added, Terraform should report no infrastructure changes.

Complete the Pull Request

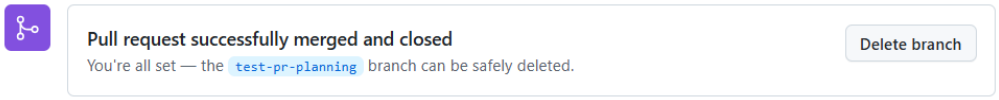
24. Return to the Pull Request and select:

    Merge Pull Request
    Confirm the merge.

25. Observe that:

    - The PR has been completed successfully
    - No infrastructure changes occurred
    - The merge represents acceptance of the proposed change

26. Delete the branch..



Checkpoint

At this point:

- The existing .github/workflows folder now contains three new PR plan workflows
- change-all.yml still exists and has not yet been retired
- Each PR plan workflow targets a single environment
- The PR plan workflows perform Terraform validation and planning only
- No deployment activity occurs
- Planning and deployment responsibilities are now separated

Phase 3 – Remove the Centralised Deployment Workflow

The change-all workflow introduced in Part 2 has now served its purpose. It demonstrated how GitHub Actions can enforce promotion order, perform Terraform deployments, and integrate with GitHub Environments for approval-based governance.

However, the workflow was intentionally designed as a centralised deployment mechanism. Regardless of which environment changed, the same workflow remained responsible for coordinating all deployments.

Now that environment-scoped planning workflows have been introduced, the repository is ready to transition towards environment-scoped deployment workflows.

Delete the Workflow

27. Locally, open **env-roots\.github\workflows**
28. Delete **change-all.yml**

Note: The original workflow remains available within **artifacts\pipelines\gitactions** and can be reviewed again at any time.

Commit and Push the Change

29. Commit the modification:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git add .\.github\workflows\change-all.yml
```

```
git commit -m "Remove centralised deployment workflow"
git push
```

Understanding the Transition

At this point, planning is environment-scoped:

pr-plan-dev
pr-plan-test
pr-plan-prod

However, deployment is temporarily absent. This gap is intentional. The centralised deployment workflow has been removed, but the environment-scoped deployment workflows have not yet been added.

In the next phase, you will introduce:

change-dev.yml
change-test.yml
change-prod.yml

These workflows will provide environment-scoped deployment capabilities and complete the transition away from the centralised deployment model.

Checkpoint

At this point:

- The centralised change-all deployment workflow has been removed
- Environment-scoped planning workflows remain active
- No deployment workflows currently exist
- The repository is ready for environment-scoped deployment workflows
- The original change-all.yml workflow remains available within the artifacts folder for future reference

Phase 4 – Add Environment-Scoped Change Pipelines

In this phase, you add dedicated change workflows for DEV, TEST and PROD. These workflows replace the centralised deployment model with environment-scoped deployment.

Add the Change Workflows

30. Review the supplied workflow files located in:

    c:\qatfadvaws\artifacts\pipelines\gitactions

    The following change workflow files have been provided:

change-dev.yml
change-test.yml
change-prod.yml

31. Copy these workflow files into: **c:\pipeline-repos\env-roots\.github\workflows**

Commit and Push the Workflows

32. Create a new branch:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D
$_.Trim() }
git checkout -b add-env-change-pipelines
```

33. Commit and push the new workflow files:

```
git add .github\workflows\change-dev.yml
git add .github\workflows\change-test.yml
git add .github\workflows\change-prod.yml
git commit -m "Add environment change workflows"
git push -u origin add-env-change-pipelines
git branch -D add-env-change-pipelines
```

Create and Merge the Pull Request

34. Open the **env-roots** repository in GitHub.
35. Create a pull request from **add-env-change-pipelines** to **main**
36. Review the files being added.
37. Merge the pull request.
38. Delete the temporary branch when prompted.

Verify the Workflows

39. Navigate to **Actions**
40. Confirm the three new workflows are visible.

Understanding the New Deployment Model

The repository now contains two workflow types.

PR plan workflows:

©QA 2026

pr-plan-dev, pr-plan-test and pr-plan-prod

These run during pull requests and stop after terraform plan.

Change workflows:

change-dev, change-test and change-prod

These run after accepted changes are merged and execute Terraform against the target environment.

The new lifecycle is:

Pull Request
↓
Environment PR Plan
↓
Review
↓
Merge
↓
Environment Change Workflow
↓
Approval if required
↓
Terraform Apply

Verify Environment-Scoped Deployment

Before continuing, verify that the new deployment workflows operate as expected.

The objective is not to deploy a meaningful infrastructure change. Instead, you will make a harmless modification that allows the new deployment model to be observed in action.

This exercise demonstrates the complete workflow lifecycle while confirming that deployment activity remains scoped to the environment being modified.

Create a Test Branch

41. Create a new branch:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D
$_.Trim() }
git checkout -b test-env-deployment
```

Create a Non-Functional Change

42. Open **envs/dev/main.tf**

43. Add the following comment anywhere within the file:

```
# Environment deployment validation test
```

Commit and Push the Change

44. Commit and push the modification:

```
git add .\envs\dev\main.tf
git commit -m "Test environment deployment workflow"
git push -u origin test-env-deployment
```

Create a Pull Request

45. Create a Pull Request from **test-env-deployment** to **main**
46. Confirm that only **pr-plan-dev** runs automatically.
47. Review the workflow output. Because only a comment was added, Terraform should report that no infrastructure changes are required.

Merge the Pull Request

48. Merge the Pull Request.
49. Delete the temporary branch when prompted.
50. Navigate to: Actions
51. Confirm that only change-dev runs automatically.
52. Review the workflow output.

    Observe:

    terraform init
    terraform validate
    terraform plan
    terraform apply

    Because only a comment was added, Terraform should report that no infrastructure changes are required.

    Although the workflow executed successfully, no resources were created, modified or destroyed.

Checkpoint

At this point:

- The PR planning workflow has been validated
- The environment deployment workflow has been validated
- A DEV change triggered only DEV workflows
- TEST and PROD remained unaffected
- Planning and deployment remain separate activities
- Environment-scoped deployment has been successfully implemented

©QA 2026

The repository now supports independent planning and deployment lifecycles for DEV, TEST and PROD.

Phase 5 – End-to-End Environment-Scoped Promotion

In this phase, you will perform a complete environment-scoped promotion from start to finish. You will:

- Create a new module version
- Pin DEV to the new version
- Open a Pull Request
- Review the DEV plan
- Merge the Pull Request
- Observe the DEV change workflow
- Verify the change

This will prove that the new model works for a real promotion, not just a comment-only test.

Release Network Module v1.0.3

53. Open the network module repository:

    c:\module-repos\terraform-aws-module-network

54. Open: **main.tf**

55. Update the module version to **v1.0.3**

    As before, this is essentially a no-op update of actual AWS resources, but it acts as a version change here. In production environments, actual resource changes would be made.

56. Tag and push the module update to Git...

    ```
    cd c:\module-repos\terraform-aws-module-network
    git status
    git add main.tf
    git commit -m "Publish network module v1.0.3"
    git tag v1.0.3
    git push origin main
    git push origin v1.0.3
    ```

57. In GitHub, open the **network** module repository and confirm that the **v1.0.3** tag exists. At this point, the module version exists, but no environment consumes it yet.

Promote Network v1.0.3 to DEV

58. Change into the env-roots repository and create a promotion branch:

```
cd c:\pipeline-repos\env-roots
git checkout main
git pull
git branch | Where-Object { $_ -notmatch "main" } | ForEach-Object { git branch -D $_.Trim() }
git checkout -b promote-network-v103-dev
```

59. Open: **envs/dev/main.tf**

60. Locate the network module source and change the version reference from:

    ?ref=v1.0.2 to **?ref=v1.0.3**

61. Commit and push the change:

```
git add .\envs\dev\main.tf
git commit -m "Promote network module v1.0.3 to DEV"
git push -u origin promote-network-v103-dev
```

Create the Pull Request

62. Open the **env-roots** repository in GitHub.

63. Create a Pull Request from **promote-network-v103-dev** to **main**

64. Review the changed file. The pull request should show a version pin change in envs/dev/main.tf from v1.0.2 to v1.0.3

Observe the PR Plan Workflow

65. Confirm that only the DEV PR plan workflow **pr-plan-dev** runs automatically

66. Open the **pr-plan-dev** workflow and review the Terraform plan. The proposed change is still only a proposal.

Merge the Pull Request

67. Once the plan has completed successfully, merge the Pull Request.

68. Delete the temporary branch. The merge represents acceptance of the proposed DEV promotion.

Observe the DEV Change Workflow

69. Navigate to **Actions**

70. Confirm that only the **DEV** deployment workflow **change-dev** starts automatically.

71. Open the **change-dev** workflow run.

72. Review the workflow output and confirm that it runs:

    terraform init
    terraform validate
    terraform plan
    terraform apply

    DEV is now be updated to consume network module **v1.0.3**

Checkpoint

At this point:

- Network module v1.0.3 has been published
- DEV has been promoted to network module v1.0.3
- The PR plan workflow showed the proposed DEV change before deployment
- The PR merge accepted the proposed change
- The DEV change workflow applied the accepted change
- TEST and PROD did not run
- TEST and PROD remain on network module v1.0.2
- The environment-scoped promotion model has been proven end-to-end

This completes the technical implementation of environment-scoped promotion pipelines.

Phase 6 – Reflection

You have now completed the transition from a centralised deployment workflow to an environment-scoped promotion model.

The objective of this part of the lab was not simply to deploy Terraform using GitHub Actions. The objective was to evolve the promotion process introduced in Lab 6A into a more scalable and auditable GitOps-style workflow.

Review the Evolution

At the beginning of Lab 6A, promotion was represented by a module version pin change:

v1.0.2 > v1.0.3

A change to the version pin represented a promotion decision.

In Part 2, promotion execution became automated through a centralised change-all deployment workflow.

In Part 3, planning and deployment became environment-scoped:

pr-plan-dev → change-dev
pr-plan-test → change-test
pr-plan-prod → change-prod

Each environment now progresses independently while preserving the promotion model established in Lab 6A.

Consider What Has Been Achieved

The solution now provides:

- Environment-scoped planning
- Environment-scoped deployment
- GitHub Environment approvals
- Independent promotion paths
- Auditable workflow execution
- Separation of planning and deployment
- Consistent Terraform execution

When reviewing a deployment, you can now identify:

- Which module version was promoted
- Which environment was affected
- Which workflow executed
- Whether approval was granted
- Whether deployment succeeded

Consider What Still Remains Unclear

Although the system now records a significant amount of information, one important question remains difficult to answer.

Consider the promotion performed in the previous phase.

You can identify:

- Version Pin Changed
- Pull Request Created
- Terraform Plan Generated
- Pull Request Merged
- Deployment Executed

However, ask yourself the following question:

Where was the promotion decision explicitly recorded?

Was the decision represented by:

© QA 2026

- The version pin change?
- The Pull Request?
- The Pull Request merge?
- The deployment approval?
- The successful workflow execution?

Each provides evidence that a promotion occurred. None explicitly records the intent behind the promotion.

The Remaining Gap

The promotion model currently relies on inference.

Reviewers must infer intent from:

Version Change
+
Pull Request
+
Merge
+
Deployment Approval

This works well for small teams.

As systems grow, organisations often require something more explicit.

Examples might include:

- Change requests
- Release records
- Promotion manifests
- Deployment authorisation records
- Formal approval documentation

These mechanisms allow organisations to distinguish between what happened and why It happened

Looking Ahead

The environment-scoped promotion model introduced in this part of the lab solves many operational challenges. However, it still leaves one important governance question unanswered: *How do we explicitly record promotion intent?*

This question becomes the focus of the next lab.

© QA 2026

Before continuing, ensure you understand not only how the workflows operate, but also the governance gap that still exists within the promotion process.

Final Checkpoint

At this point:

- Promotion remains version-driven
- Planning is environment-scoped
- Deployment is environment-scoped
- GitHub Environments provide approval controls
- Promotion execution is fully automated
- Deployment history is auditable
- Promotion intent is still inferred rather than explicitly declared

The next lab explores how organisations address this final gap.

***** Congratulations, you have completed this lab *****

©QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

©QA 2026